

FitDash

Training Copilot — System Architecture

AISS2 • Team 6 • Karlsruhe Institute of Technology

June 22, 2026

FitDash (the *Training Copilot*) is a sports-analytics assistant that unifies an athlete's Strava activities, Garmin wearable metrics, live weather, routing and calendar behind a single agentic chat. Its defining rule is that **every answer is grounded in data fetched live from real tools** — nothing is invented or cached-summarised. This document describes the key architecture: a multi-agent chat engine layered on top of a uniform, tool-agnostic data plane.

The load-bearing idea: MCP-standard and tool-agnostic

One uniform client talks to many independent servers; tools are *discovered*, *never hard-coded*; and authentication is separated from tool declarations. Adding a data source is one new server file plus one registry line — no change to the host, the agents, or the UI. Internalising this explains where every component lives.

1 System overview

The system is a vertical stack of well-separated layers. A user request enters at the top, is decomposed by an orchestrator agent, delegated to specialist agents, and resolved against external data through a single tool-calling seam. Two paths run through it: the *chat path* (UI → orchestrator → specialists) and the *data path* (agent → ToolHost → MCP servers → external APIs).

2 Core seams (core/)

`core/` is deliberately Streamlit-free and vendor-neutral, so it can run standalone behind any front-end. Four small modules carry the architecture:

`config.py` — the registry. A flat name → URL dictionary (MCP_SERVERS) plus the A2A agent registry (A2A_AGENTS) and the per-agent tool scope (AGENT_MCP_SCOPE). Every URL is environment-overridable. Tool names are namespaced `server__tool` (the separator `SEP="__"`, since dots are illegal in OpenAI function names).

`host.py` — ToolHost. The single MCP client for the entire app. `list_tools()` discovers every tool from every *reachable* server in OpenAI-tool format; an unreachable or unauthorised server is silently skipped, never fatal. `call_tool("server__tool", args)` splits the name, routes the call, and returns text/JSON — tool errors come back as `{"error": ...}` strings, not exceptions. The implementation is async with a synchronous facade for thread/Streamlit callers.

`llm.py` — the LLM seam. The only place a chat client is constructed and the model resolved, both from `.env`. `LLM_PROVIDER` switches the whole app between `openai` (KIT gateway or any OpenAI-compatible endpoint), `openai_official` (`api.openai.com`), and `gemini` — a configuration change, not a code change. The seam re-reads `.env` per call, so a model switch in the Settings UI applies without restart.

`orchestrator.py` — FitDashOrchestrator. A thin synchronous A2A *client adapter* to the orchestrator agent. It preserves one stable public contract for all callers — `run(user_input, history, progress_cb, text_cb) → (answer, trace)` and `refresh_tools()` — so the React seam, the Streamlit chat, and the Telegram bridge are interchangeable front-ends over the same engine.

3 Agent layer — LangGraph specialists over A2A

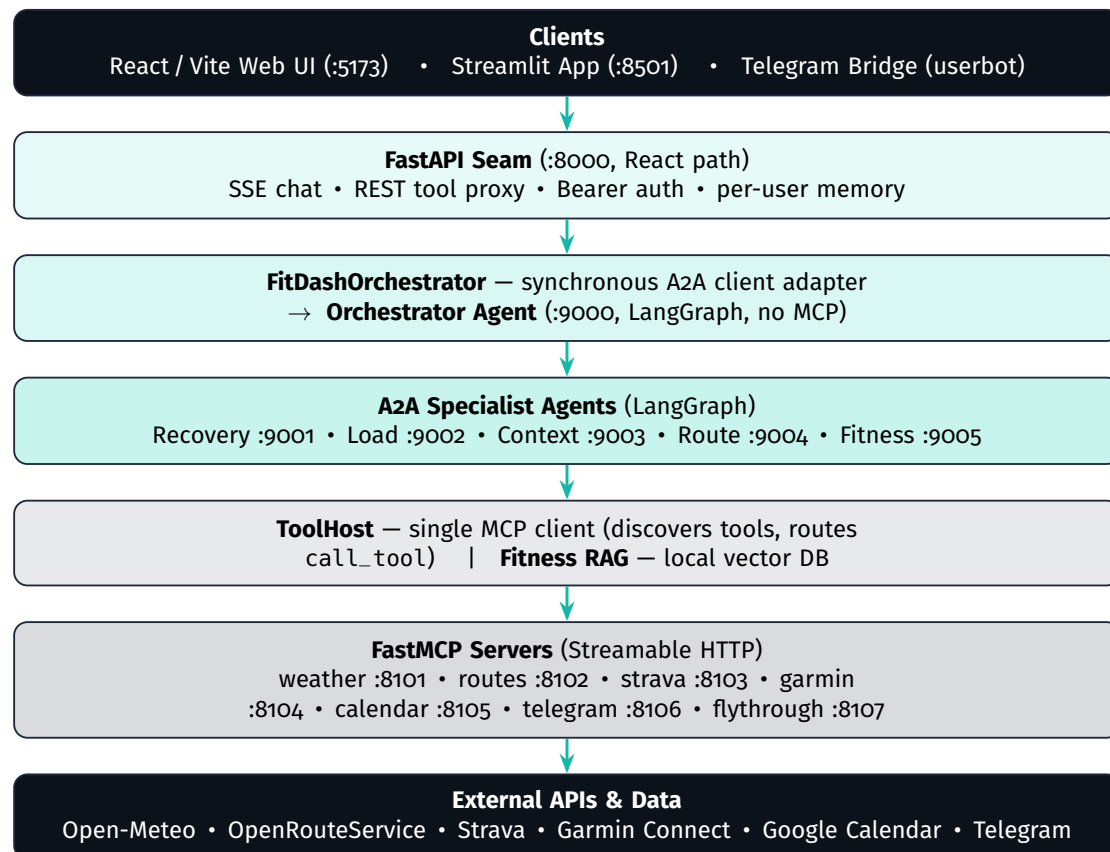


Figure 1: Layered architecture. The React UI reaches the orchestrator through the FastAPI seam; the Streamlit app and the Telegram bridge embed FitDashOrchestrator in-process. All upstream data access funnels through ToolHost.

The chat engine is a multi-agent system. Each agent is its own **A2A server** (the official `a2a-sdk`, exposing an Agent Card at `/well-known/agent-card.json`), so agents are independently deployable and discoverable.

Orchestrator (:9000). A LangGraph agent whose only tools are `ask_*` — one per specialist, each performing an A2A call. It decomposes the user request, delegates (in *parallel* when the model emits several tool calls at once), collects each specialist's `DataPart` artifact, and assembles the structured trace via `core/agent_trace.build_trace`. It has **no MCP access** of its own — it only coordinates.

Specialists. Each is a LangGraph ReAct agent over a ToolHost scoped to a narrow set of MCP servers. Tools remain discovered, never hardcoded — merely narrowed per agent. Specialists return their raw MCP results (full JSON) as artifacts so the orchestrator can build route maps, charts, and the debug trace.

Fitness Expert — the RAG exception. The fitness agent has no MCP server. It answers exercise-science questions from a local vector database of public-domain fitness books (`core/fitness_rag.py`). Its single `search_fitness_literature` tool is recorded in the same artifact shape as any MCP call, so it flows into the trace uniformly. Embeddings use a small *local* model (`sentence-transformers`, `all-MiniLM-L6-v2`) — no embedding API.

4 Data plane — MCP servers & integrations

Agent	Port	MCP scope	Responsibility
Orchestrator	9000	—	Decompose request, delegate in parallel, synthesise answer, build trace
Recovery	9001	garmin	Sleep, HRV, Body Battery, stress → readiness / should I rest?
Load	9002	strava, garmin	Training load & trends (CTL/ATL/TSB), PBs, pace, HR zones, activity detail
Context	9003	weather, calendar	Weather forecast × calendar free time → best training window; writes calendar events
Route	9004	routes	Plan routes / loops / park loops / trails, geocode places, elevation, isochrones
Fitness	9005	— (RAG)	Training methods & technique answered from a vector DB of fitness literature, with citations

Table 1: The orchestrator and the five specialists. Scope comes from `AGENT_MCP_SCOPE`; unreachable specialists degrade gracefully rather than failing the turn.

Each data source is a self-contained **FastMCP** server over Streamable HTTP. There is no shared base class and no dispatch indirection: a tool calls its upstream API directly and returns clean JSON. The `@mcp.tool()` docstring is the tool's interface — it is what the model reads to choose a tool.

Server	Port	Upstream / role
weather	8101	Open-Meteo — forecast, current conditions, UV, pollen
routes	8102	OpenRouteService — routing, circular & park loops, trails, elevation, isochrones
strava	8103	Strava — activities, training trends, personal bests, streams
garmin	8104	Garmin Connect — sleep, HRV, Body Battery, stress, training metrics
calendar	8105	Google Calendar — read and write events
telegram	8106	Proxy bridging the external chigwell/telegram-mcp (stdio → HTTP)
flythrough	8107	3D GPS-track flythrough render support

Table 2: The MCP server registry (`MCP_SERVERS`). Credentials are passed as connection headers via `ToolHost(headers=...)` — never as tool arguments, never in model context.

The **Telegram** server is the one proxy rather than a native server: it runs the upstream stdio-only MCP project unmodified in an isolated environment and re-exposes its tools on the Streamable-HTTP bus, holding one persistent upstream session. To `ToolHost` it looks identical to every other server — the template for bridging any external stdio MCP server into the app.

5 Request lifecycle

1. A user asks a question in the React chat, the Streamlit app, or over Telegram. The client calls `FitDashOrchestrator.run(question, history)` — in-process (Streamlit, Telegram) or through the FastAPI SSE endpoint (React).
2. `run()` flattens the history into one A2A message and sends it to the Orchestrator Agent (:9000).
3. The orchestrator (LangGraph) decomposes the request and invokes one or more `ask_*` tools — each an A2A call to a specialist — *in parallel* when the sub-questions are independent.
4. Each specialist discovers the tools of its scoped MCP servers via `ToolHost` and calls the ones it needs (the Fitness agent retrieves from its RAG store instead). All upstream access goes through `call_tool`; failures return an `{"error"}` payload rather than throwing.
5. Specialists return raw results as A2A `DataPart` artifacts. The orchestrator synthesises the final answer and assembles a structured `trace` (tool calls, agents, route data, timings).
6. The answer streams back to the client; the UI renders inline route maps and charts from the

trace. Every agent run is traced to MLflow (Section 6).

6 Observability — MLflow tracing

Every agent run is traced to an MLflow tracking server (UI on :5001). `core/tracing.py` is the seam: each agent process enables `mlflow.langchain.autolog()` once and wraps its invocation in a labelled root span tagged with its service (orchestrator / recovery / load / context / route). The autologged LLM and tool spans nest beneath that root, so one tagged trace per run lands in the shared `fitdash` experiment — filterable by service, with each `ask_*` delegation and each MCP tool call as a child span. Tracing is best-effort: a missing or unreachable MLflow server logs once and is ignored, and the agents run untraced.

7 Clients, entry points & cross-cutting concerns

React / Vite web app (`web/`). The primary UI (Dashboard, Health, Routes, Analysis, Chat, Sync, Settings). It reaches the engine through the **FastAPI seam** (`api/`), which exposes SSE chat, a REST tool proxy, settings, sync, and memory.

Streamlit app (`app.py`). The legacy UI, embedding `FitDashOrchestrator` directly in-process.

Telegram bridge (`telegram_bridge.py`). A userbot that forwards each message (and locally transcribed voice memos) to the same orchestrator and returns the answer, delivering routes as a static map, a Google Maps link, and a GPX file.

Identity & auth. The FastAPI seam authenticates with stateless HMAC Bearer tokens (`api/auth.py`); protected routers depend on the current user.

Per-user memory. Each user has a self-updating “soul” profile (Markdown the agent periodically distils from the conversation) plus a per-user conversation vector store for exact recall (all-MiniLM-L6-v2), blended into the orchestrator prompt.

8 Extensibility & technology stack

The architecture optimises for one operation: **adding a data source**. Write a FastMCP server file with `@mcp.tool()` functions, add one line to `MCP_SERVERS`, start the process — and the chat agents can call it immediately. No change to the host, orchestrator, or UI. Adding a *specialist* is symmetric: one A2A agent process plus one registry entry.

Layer	Technology
LLM seam	LangChain ChatOpenAI / Gemini; provider via <code>LLM_PROVIDER</code>
Agents	LangGraph (<code>create_agent</code> / <code>ReAct</code>) over <code>a2a-sdk</code> (A2A protocol, agent cards)
Tools / data	MCP — FastMCP servers (Streamable HTTP) behind a single <code>ToolHost</code>
Web front-end	React + Vite + TypeScript + Tailwind (<code>web/</code>); FastAPI seam (<code>api/</code>)
Legacy UI	Streamlit (<code>app.py</code>)
Messaging	Telethon userbot (<code>telegram_bridge.py</code>)
RAG	sentence-transformers all-MiniLM-L6-v2 + local NumPy vector store
Observability	MLflow tracing (autolog + tagged spans)

Table 3: Technology stack by architectural layer.